Pointer Refresher

Revisiting Core Concepts and Operations

---

1. Basic Pointer and Dereferencing

```c
#include <stdio.h>

int main() {
    int a = 10;
    int *p = &a;

    printf("a: %d\n", a);
    printf("*p: %d\n", *p);
    printf("p: %p\n", p);
    printf("&a: %p\n", &a);

    return 0;
}
```

---

2. Pointer and Array Relationship

```c
#include <stdio.h>

int main() {
    int arr[3] = {1, 2, 3};
    int *p = arr;

    printf("%d\n", *p);
    printf("%d\n", *(p + 1));
    printf("%d\n", *(p + 2));

    return 0;
}
```

```c
#include <stdio.h>

int main() {
    int arr[5] = {1, 2, 3, 4, 5};

    for (int i = 0; i < 5; i++) {
        printf("Address using &arr[%d]: %p\n", i, &arr[i]);
        printf("Address using arr + %d: %p\n", i, arr + i);
    }

    return 0;
}
```

3. Pointer Increment vs Dereferencing

```c
#include <stdio.h>

int main() {
    int arr[] = {5, 10, 15};
    int *p = arr;

    printf("%d\n", *p++);  // Post-increment pointer
    printf("%d\n", *p);    // After pointer increment

    return 0;
}
```

---

4. Pointer Arithmetic

```c
#include <stdio.h>

int main() {
    int arr[] = {10, 20, 30, 40};
    int *p = arr;

    printf("%d\n", *p);     // Pointing to first element
```

```
    p++;
    printf("%d\n", *p);     // After incrementing pointer
    p += 2;
    printf("%d\n", *p);     // After incrementing by 2

    return 0;
}
```

Q. Accessing Array Elements Using Pointers

```
#include <stdio.h>

void printArray(int *arr, int size) {
    // Traverse array using pointer
}

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int size = sizeof(arr) / sizeof(arr[0]);

    printArray(arr, size);

    return 0;
}
```

---

5. Changing Values through Pointers

```
#include <stdio.h>

int main() {
    int x = 5;
    int *ptr = &x;

    ptr = 15;
    printf("%d\n", x);

    return 0;
}
```

Q. Modifying Array Elements Using Pointers

- Double each element in the given array.

```c
#include <stdio.h>

void doubleValues(int *arr, int size) {
     for(int i = 0; i < size; i++)
      {
      *(arr + i) = 2* (*(arr + i));
//arr[i] = arr[i] *2;
      }
}

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int size = sizeof(arr) / sizeof(arr[0]);

    doubleValues(arr, size);

    // Print modified array

    return 0;
}
```

Q. Swapping Elements of an Array Using Pointers

- Swap the first and last element.

```c
#include <stdio.h>

void swap(int *a, int *b) {
    // Swap the values
}

void swapFirstLast(int *arr, int size) {
    // Call swap to swap first and last element
}

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int size = sizeof(arr) / sizeof(arr[0]);
```

```
    swapFirstLast(arr, size);
    // Print modified array

    return 0;
}
```

//code for swapping two elements

#include <stdio.h>

```
void swap(int* var1, int* var2)
{
int temp = *var1; // temp --> 10
*var1 = *var2; // var1 (&a) --> Address of b
*var2 = temp; //var2 --> Address of a
}


int main() {
      int a = 10;
      int b = 20;
      swap(&a,&b);
      printf("%d ",a);
      printf("%d",b);
      return 0;
}
```

Q. Reversing an Array Using Pointers [Homework Problem]

```
#include <stdio.h>

void reverseArray(int *arr, int size) {
    // Use two pointers
}

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int size = sizeof(arr) / sizeof(arr[0]);

    reverseArray(arr, size);
```

```
    // Print reversed array

    return 0;
}
```

---

6. Multiple Pointers

```
#include <stdio.h>

int main() {
    int a = 50;
    int *p1 = &a;
    int **p2 = &p1;

    printf("%d\n", a);
    printf("%d\n", *p1);
    printf("%d\n", **p2);

    return 0;
}
```

Pointer to a 2D Array

- **Printing a 2D array**

```
#include <stdio.h>

void print2DArray(int (*arr)[3], int rows) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < 3; j++) {
            printf("%d ", *(*(arr + i) + j));  // Pointer to array
        }
        printf("\n");
    }
}

int main() {
    int arr[2][3] = {{1, 2, 3}, {4, 5, 6}};
```

```
    print2DArray(arr, 2);

    return 0;
}
```

Q. You are given a set of N 2D points (samples), where each point is represented as a pair of coordinates (x,y) in a 2D plane. The coordinates of the points are stored in a 2D array.

Write a C program to find the closest neighbor to a given target point from the set of N points. The distance between two points can be calculated using Euclidean distance.

[Homework Problem]

```
#include <stdio.h>
#include <math.h>

int closestNeighbor(int points[][2], int n, int target[2]) {
    // Initialize variables to keep track of the closest point and its
index

    // Iterate through all points

        // Calculate the Euclidean distance between the target and the
current point

        // Update the closest point if necessary

    // Return the index of the closest point
}

int main() {
    int points[4][2] = {{1, 2}, {3, 4}, {5, 1}, {2, 1}};
    int target[2] = {2, 3};

    int index = closestNeighbor(points, 4, target);

    printf("The closest point is at index: %d\n", index);

    return 0;
}
```

Q. [Medium Difficulty] Find the k-closest neighbors to a target point.

Dynamic Memory Allocation with Arrays and Pointers

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int size;
    printf("Enter the size of the array: ");
    scanf("%d", &size);

    int *arr = (int *)malloc(size * sizeof(int));

    printf("Enter the elements of the array:\n");
    for (int i = 0; i < size; i++) {
        scanf("%d", arr + i);
    }

    printf("Array elements are:\n");
    for (int i = 0; i < size; i++) {
        printf("%d ", *(arr + i));
    }
    printf("\n");

    free(arr);

    return 0;
}
```